



Tema 2 - Fases del compilador

Cátedra: Organización del Computador II
Lic. Juan Pablo Moreno

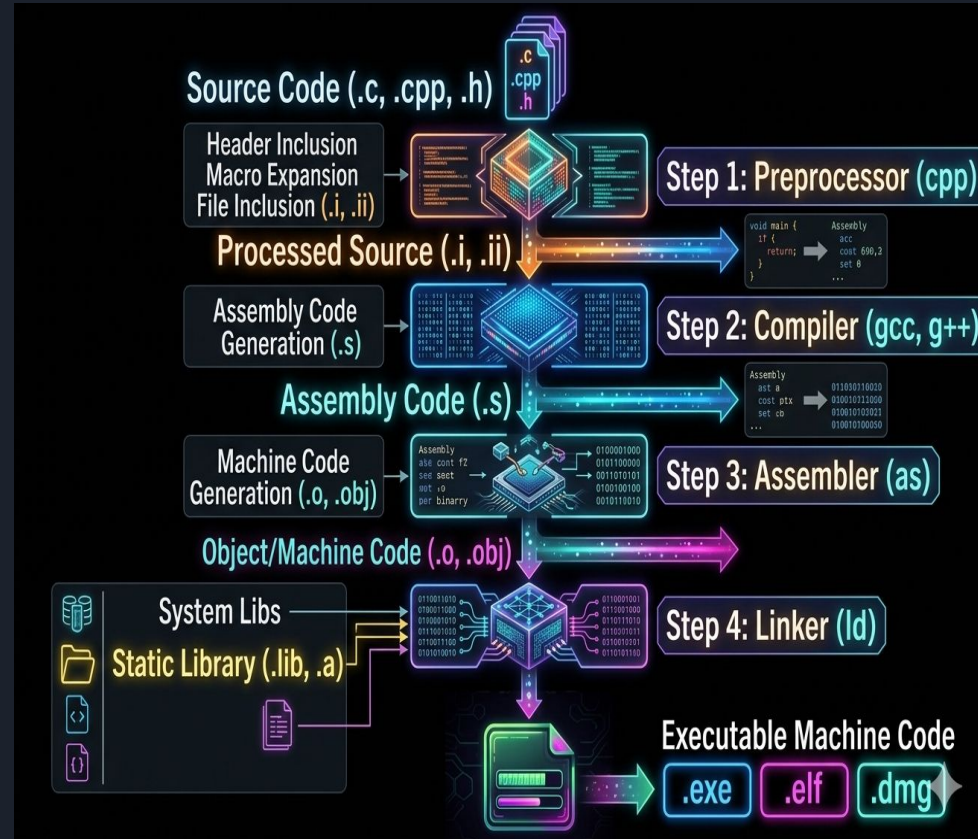


Contenido

1. Etapas de compilación
2. Preprocesamiento y Compilación
3. Scanner
4. Parser
5. Análisis Semántico
6. Traducción
7. Lenguaje de Representación Intermedia
8. Generación de Código Intermedio
9. Optimizador Independiente
10. Generación y Mejora de Código Objetivo
11. Estrategias de Optimización

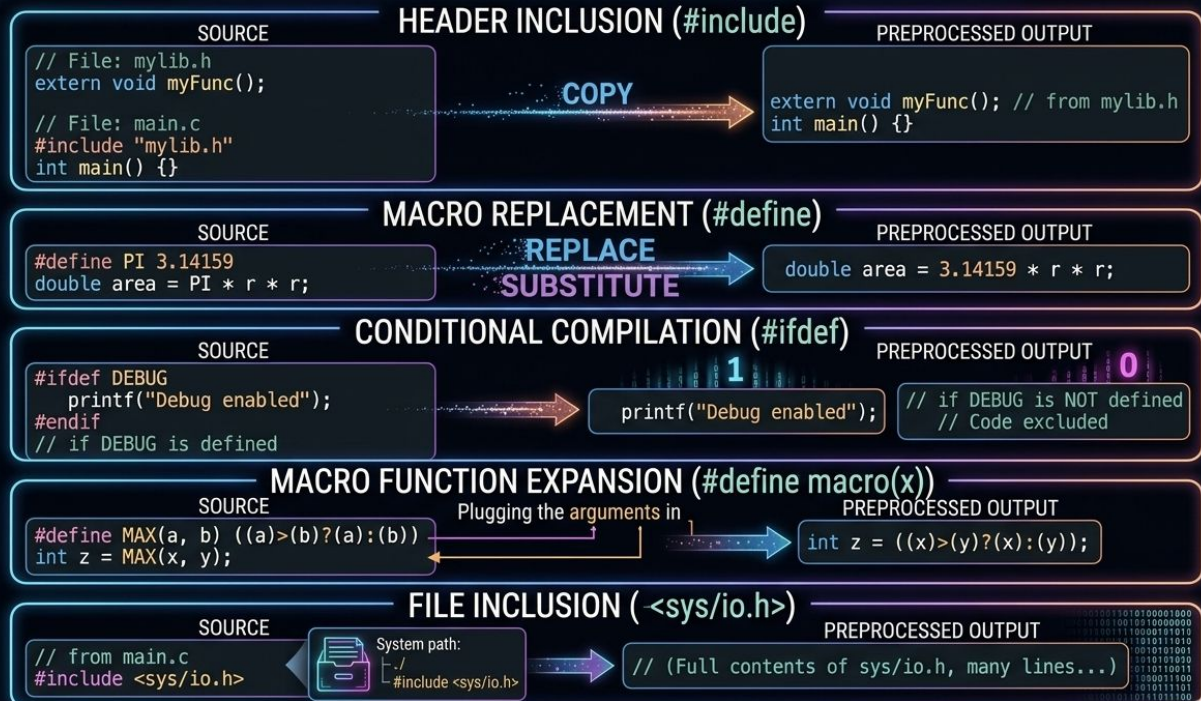
Etapas de compilación

- Preprocesamiento: Prepara el código
 - #Include #define
 - Elimina comentarios
- Compilación
 - Convierte a .asm
- Ensamblaje
 - Lo Convierte en un archivo Objeto
- Vinculación
 - Se buscan las llamadas a funciones utilizadas en nuestro código
 - Se Reemplazan por las definiciones de cada función
- Carga y Ejecución



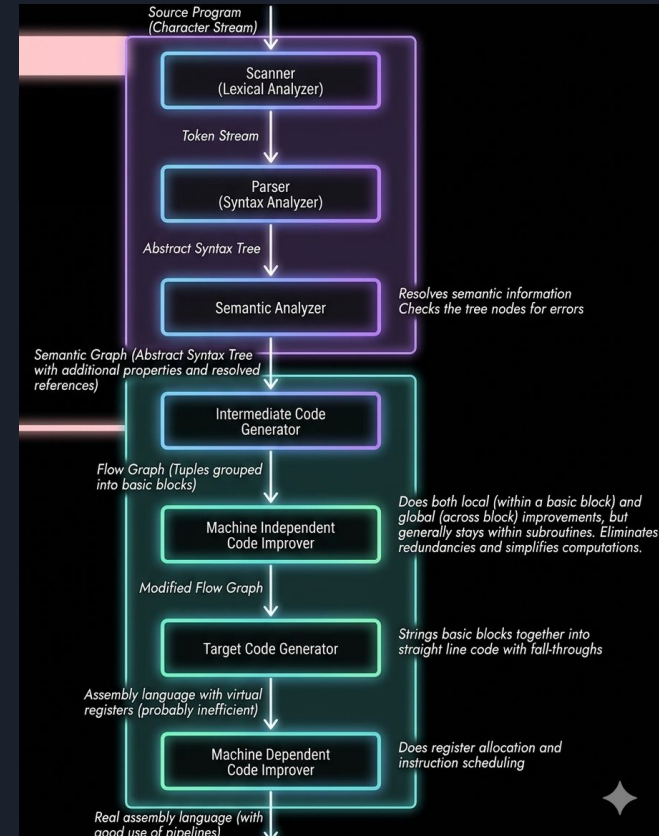
Etapas de compilación

C/C++ PREPROCESSOR OPERATIONS: 5 EXAMPLES



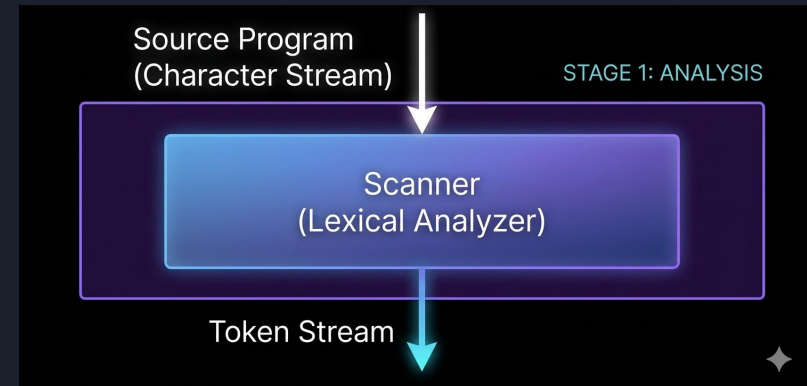
Preprocesamiento y Compilación

- Lo que parecían dos etapas en verdad es un proceso aún más grande
- Está comprendido de pequeñas etapas cada una con un propósito pequeño
 - Scanner (Análisis Léxico)
 - Parser (Análisis de Sintaxis)
 - Análisis Semántico
 - Generador de Código Intermedio
 - Optimizador Independiente
 - Generador de código Objetivo
 - Optimizador Objetivo



Scanner

- Convierte código en tokens (etiquetas)
- Mientras lo realiza:
 - Elimina comentarios
 - Expande Macros `#define` `#include`
 - Elimina espacios en blanco
- Errores que detecta:
 - Caracteres que no pertenecen al lenguaje
 - Strings o caracteres sin cerrar
 - Más caracteres que los permitidos
xej `char a = 'abc'`



Scanner

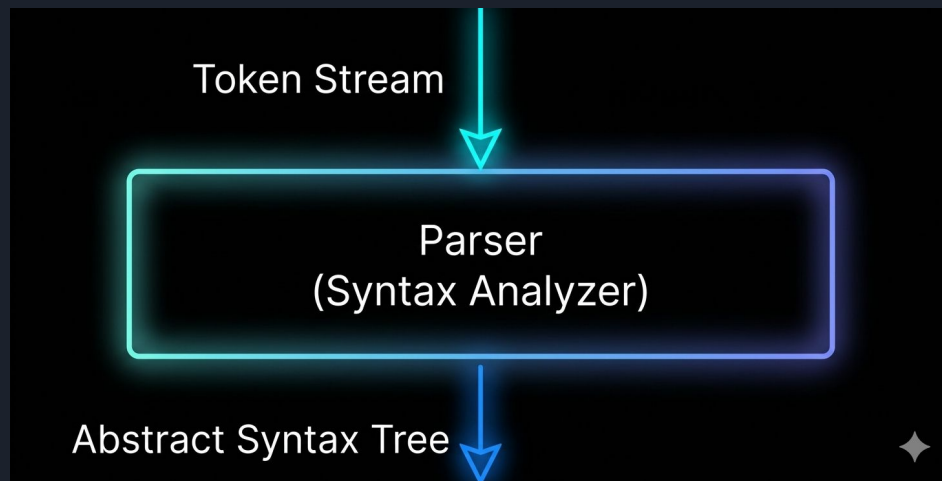
```
#define ZERO 0
    unsigned gcd(    unsigned    int // Euclid's algorithm
        x,unsigned    y) {    while ( /* hello */    x> ZERO
    ){unsigned temp=x;x=y    %x;y    = temp ;}return y ;}
```

unsigned ID(gcd) (unsigned int ID(x) , unsigned ID(y)) { while (ID(x)
 > INTLIT(0)) { unsigned ID(temp) = ID(x) ; ID(x) = ID(y) % ID(x) ;
 ID(y) = ID(temp) ; } return ID(y) ; }

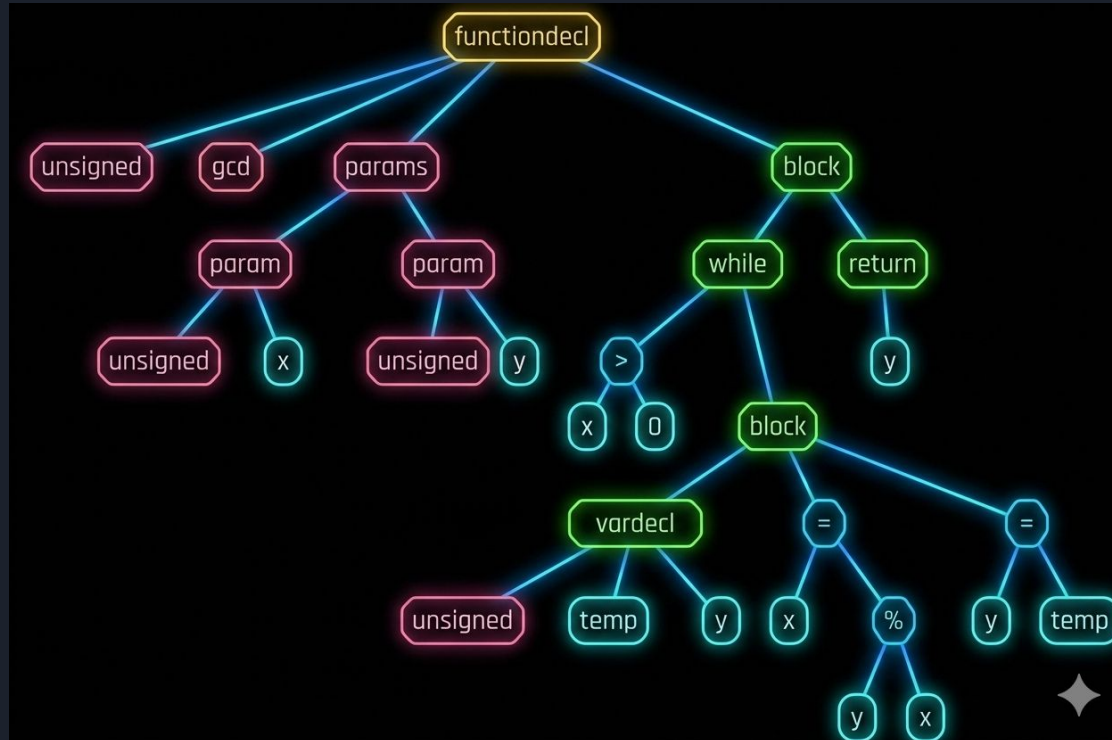


Parser

- Convierte tokens en árbol de sintaxis abstracto
- No contiene ciclos
- Errores que detecta (de estructura):
 - $j = 4 * (6 - x;$
 - $i = /5$
 - $42 = x * 3$
- Cada tipo de bloque es un objeto que contiene a otros objetos

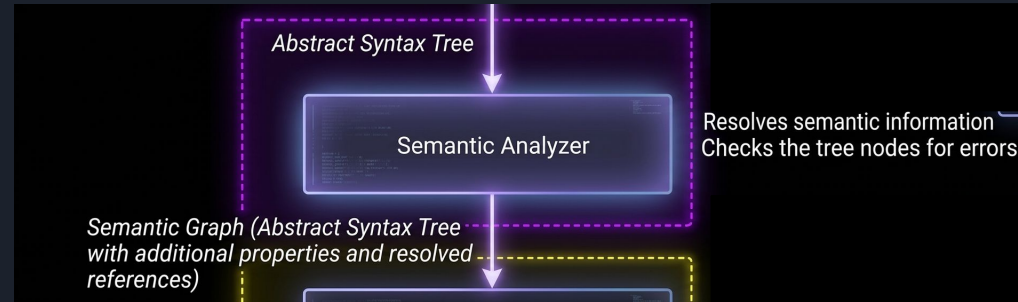


Parser

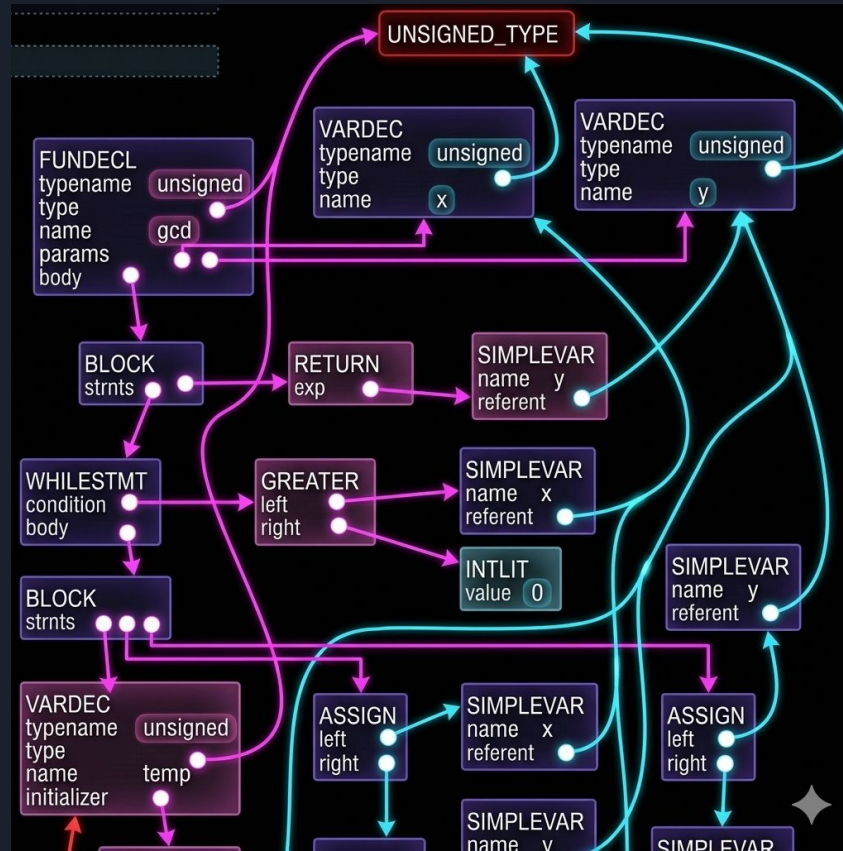


Análisis Semántico

- La semántica le da significado a una construcción
 - Nuestro Árbol abstracto tenía estructura pero no significado
- Agregamos ciclos al árbol para darle significado
- Esta etapa detecta errores como:
 - Error de tipos
 - Variables sin declarar o con múltiples declaraciones
 - Violación de reglas de acceso (private, public, protected)
 - Desfase entre argumentos de un método y su definición



Análisis Semántico



Traducción

Traducir es pasar de un lenguaje al otro

Hacer “match” entre un idioma y el otro directamente sin entender su significado trae problemas

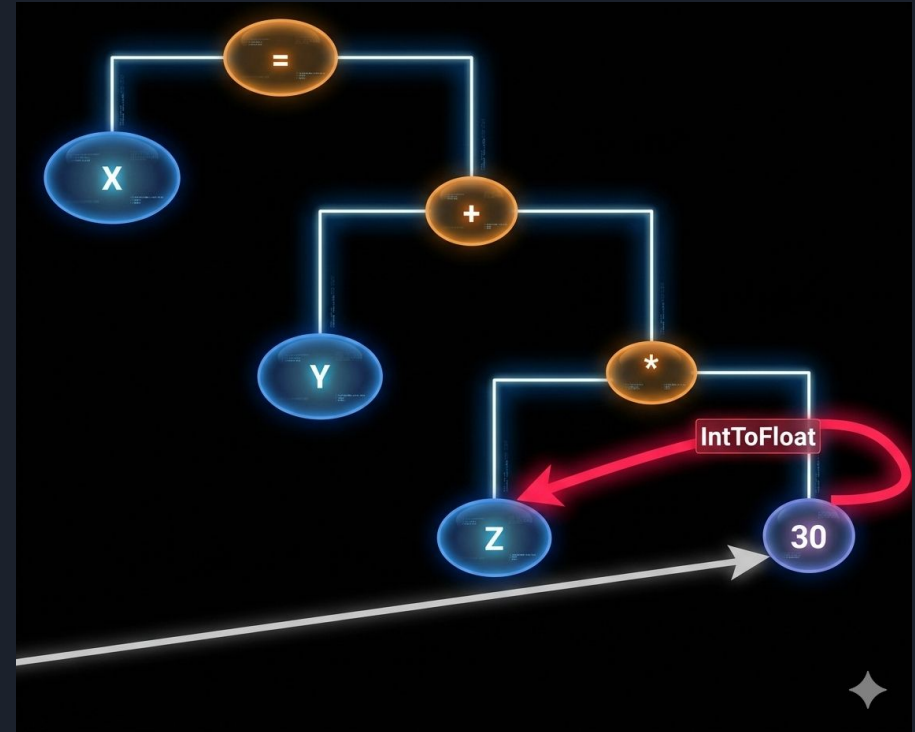
- Por ejemplo, frases hechas del Español al Inglés o al revés:
- Tomar el pelo → Take the hair? (Pull someone's leg)
- Tirar la casa por la ventana → Throw the house through the window?
- Piece of cake → ¿Pedazo de torta? (Pan comido)



Ejemplo

Código fuente: $X = Y + Z * 30$

- **Análisis Léxico**
 - Aislo tokens o lexemas y expresiones del lenguaje
 - Los guardo en la tabla de símbolos de la forma $\langle \text{Símbolo}, \text{ID} \rangle$ o $\langle \text{Símbolo} \rangle$
 - $\langle X, 1 \rangle, \langle = \rangle, \langle Y, 2 \rangle, \langle + \rangle, \langle Z, 3 \rangle, \langle * \rangle, \langle 30 \rangle$
- **Análisis Sintáctico**
 - Chequea que la estructura se ajuste a las reglas del lenguaje
 - Construyo árbol de sintaxis abstracto
 - Descarto lo que no es sintácticamente correcto
- **Análisis Semántico**
 - Decoro el árbol casteando los tipos y otras tareas que otorgan contexto



Lenguaje de Representación Intermedia

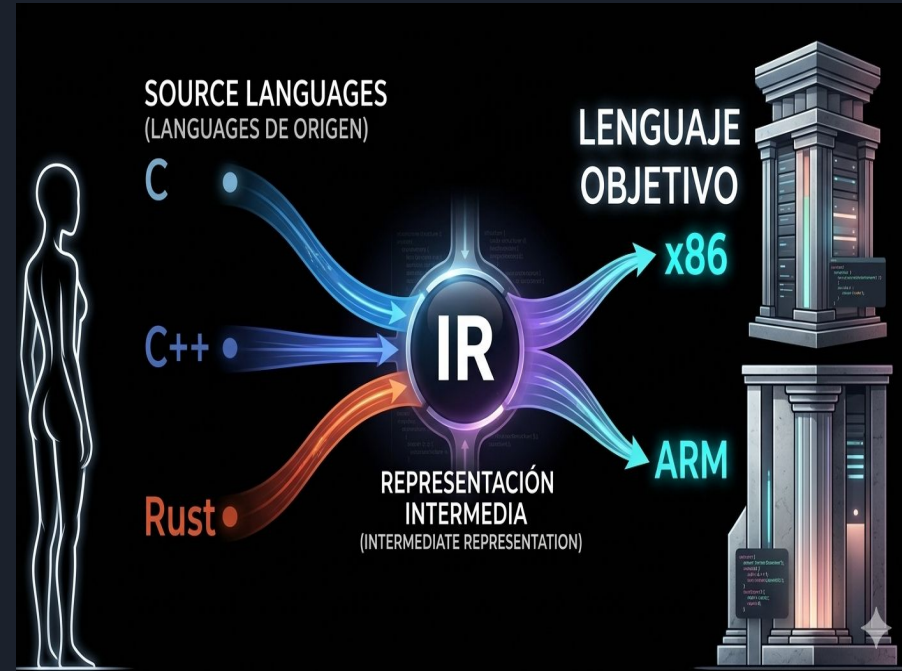
Es un lenguaje que se para en la mitad entre el código objeto y el fuente y tiene lo menos posible de ambos.

Traducir requiere de análisis y de síntesis, conviene descomponer el proceso de traducción en dos etapas desacopladas

Si hiciera todo junto necesitaría $N(\text{Lenguajes}) * M(\text{Arquitecturas})$ cantidad de compiladores

Sólo escribo el analizador para cada lenguaje de alto nivel y el generador para cada arquitectura

Permite realizar optimizaciones independientes de la arquitectura



Generador de Código Intermedio

Basado en tuplas

Puede ser de distintos niveles:

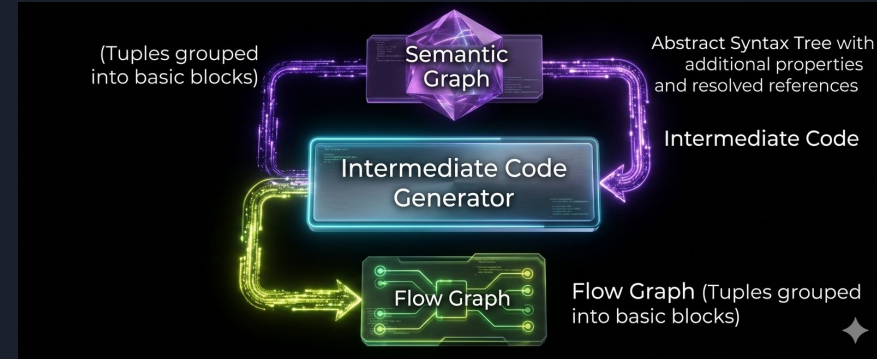
Alto nivel → Se parecen mucho al código fuente, poseen arreglos y otros objetos estructurados

Nivel medio → No hay objetos estructurados pero sigue siendo independiente del lenguaje Objetivo

Bajo nivel → Los operandos están muy cerca de el lenguaje objetivo

Puede ser representado con un Gráfico de control de flujo.

Las aristas son transiciones entre bloques (nodos)



Entre muchos se destacan:

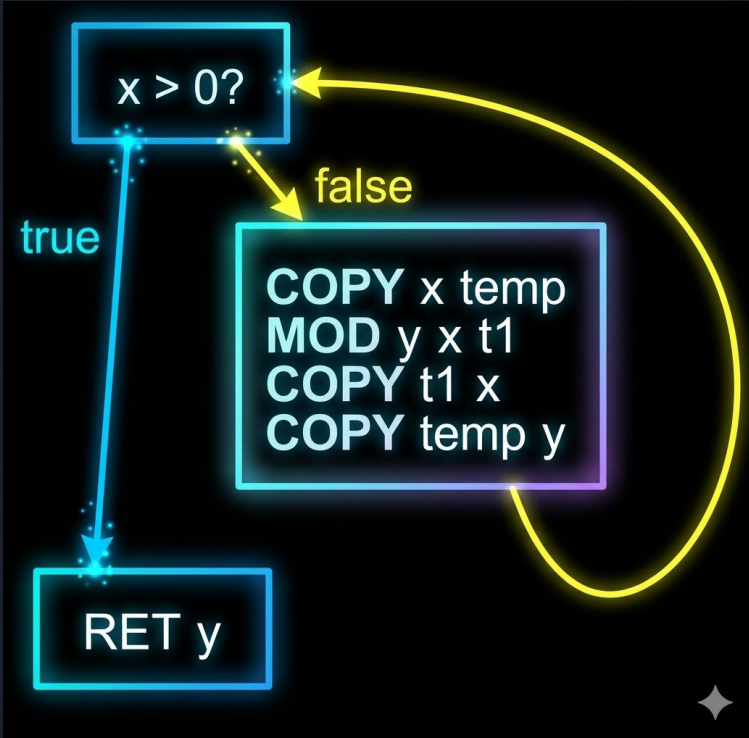
- LLVM IR
- C--
- asm.js
- C
- Java bytecode

Generador de Código Intermedio

```

LABEL top
GT x 0. t0.
JZ t0. bottom
COPY x temp
MOD y x t1.
COPY t1. x
COPY temp y
JUMP top
LABEL bottom
RET y

```



Optimizador Independiente

Optimizar código mejora el tamaño, la velocidad y el uso de recursos de un programa

Ya que muchas tareas de optimización son de complejidad exponencial o son NP-Hard. Normalmente hablamos de mejora en vez de optimización

Aplica estrategias de optimización que son independientes de la arquitectura para la que se compila (una ventaja de tener IR).

Estrategias de optimización:

- Reducir la cantidad de código
- Eliminar la cantidad de saltos (Quitarle trabajo al branch predictor)
- Reusar resultados en vez de re-computarlos



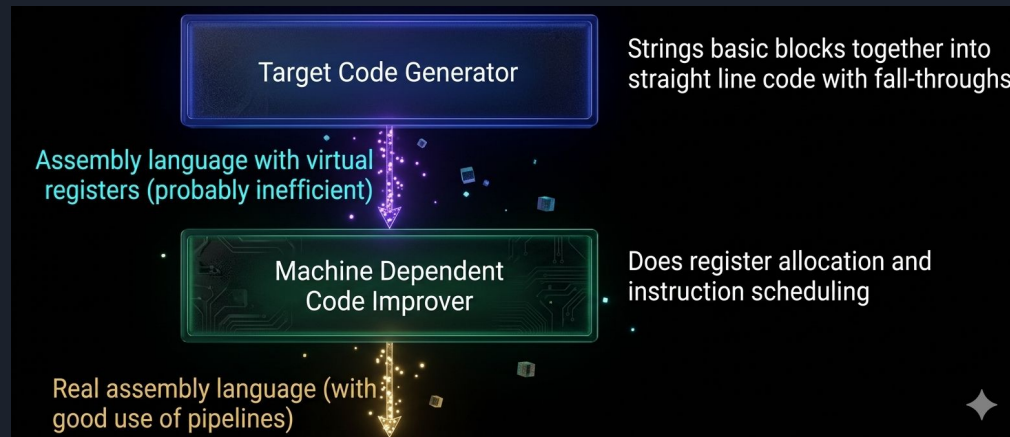
Generación y Mejora de Código Objeto

Mejoras que dependen de si la arquitectura objetivo las soporta

Tienen que ver con hacer buen uso del hardware

Assembler en su estado final Aprovecha:

- Pipelines
- Reserva Registros
- Tamaño de caché
- Aprovecha bien si es posible los núcleos del CPU



Estrategias de Optimización

Optimizaciones Independientes

- **Constant Folding**
- Algebraic Simplification (e.g. operand reordering) 
- Unreachable Code Elimination 
- Dead Code Elimination 
- Tail Recursion Elimination 

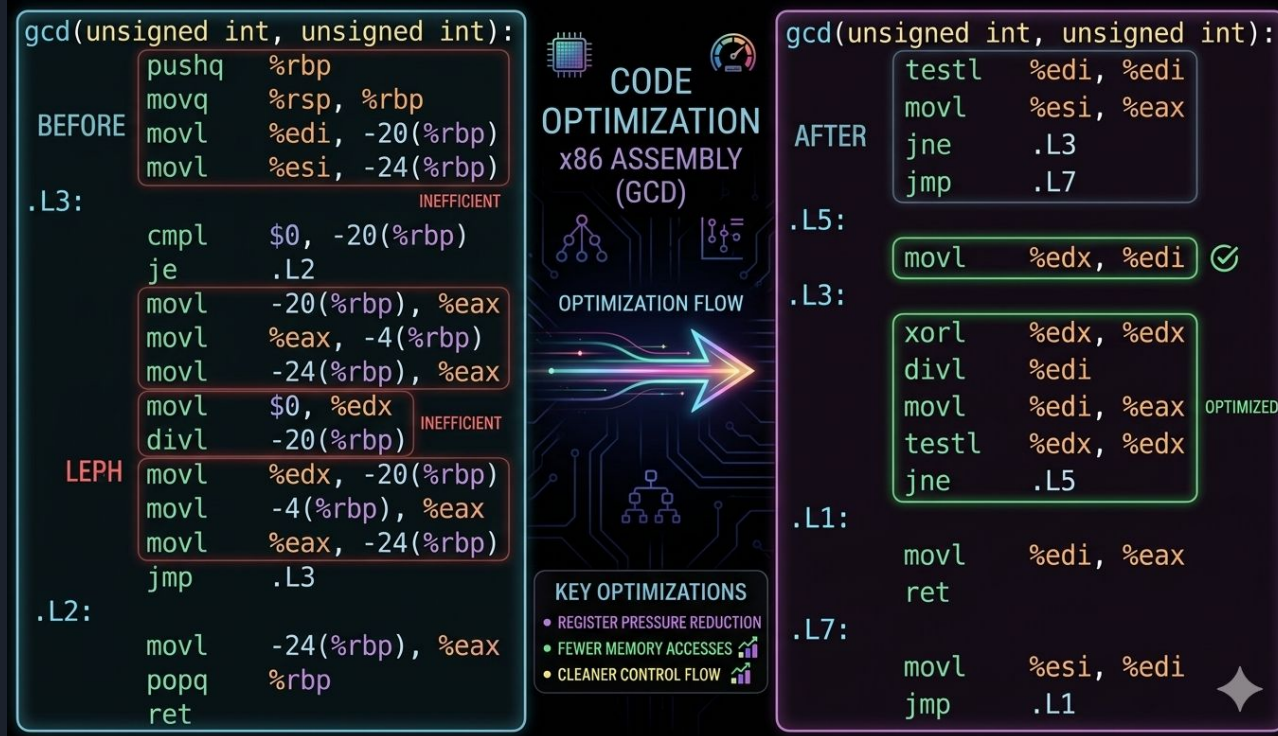
Optimizaciones "Casi" Independientes

- Efficient address computation 
- Simplified stack frames 
- Static allocation of frames
- Inlining 
- **Loop Unrolling**

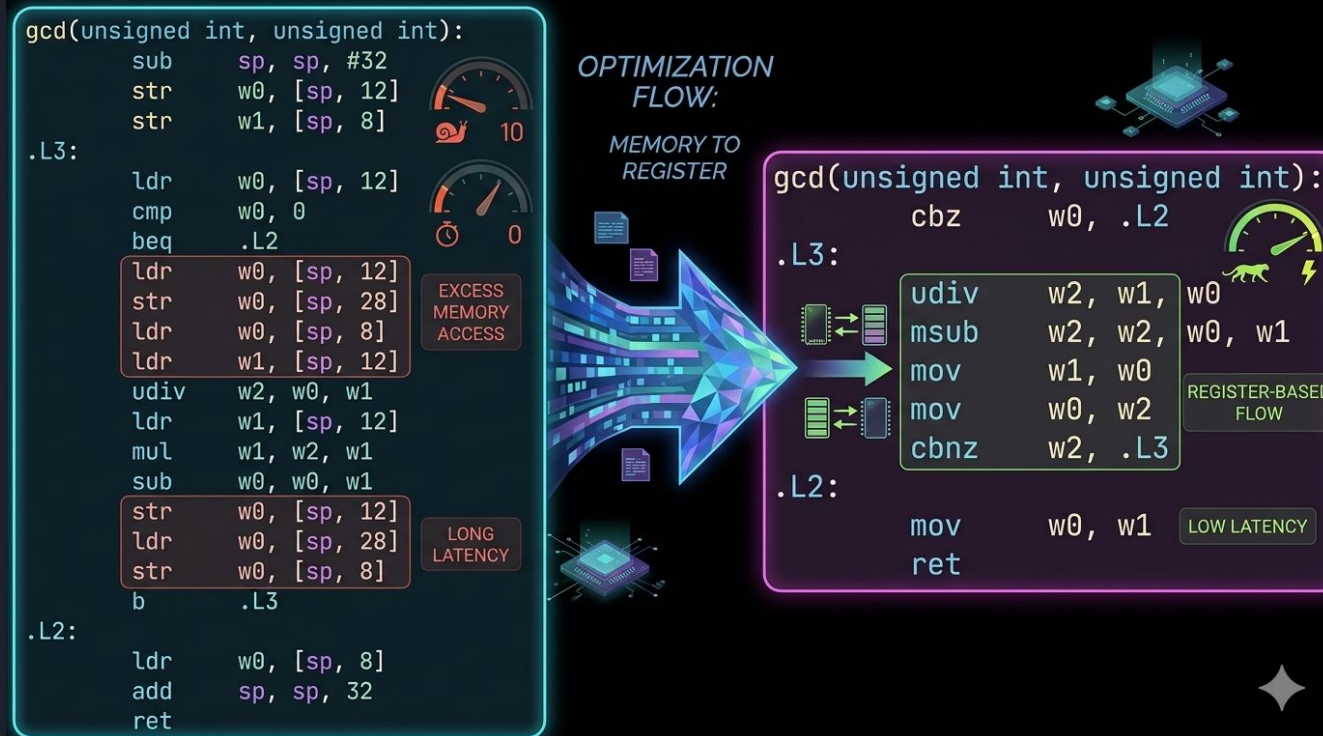
Optimizaciones No Independientes

- Careful instruction selection, using an understanding of caches, pipelines, branch prediction algorithms, scheduling rules, alignment requirements 
- **Smart register allocation**
- Smart use of address modes
- Balancing across multiple CPUs 

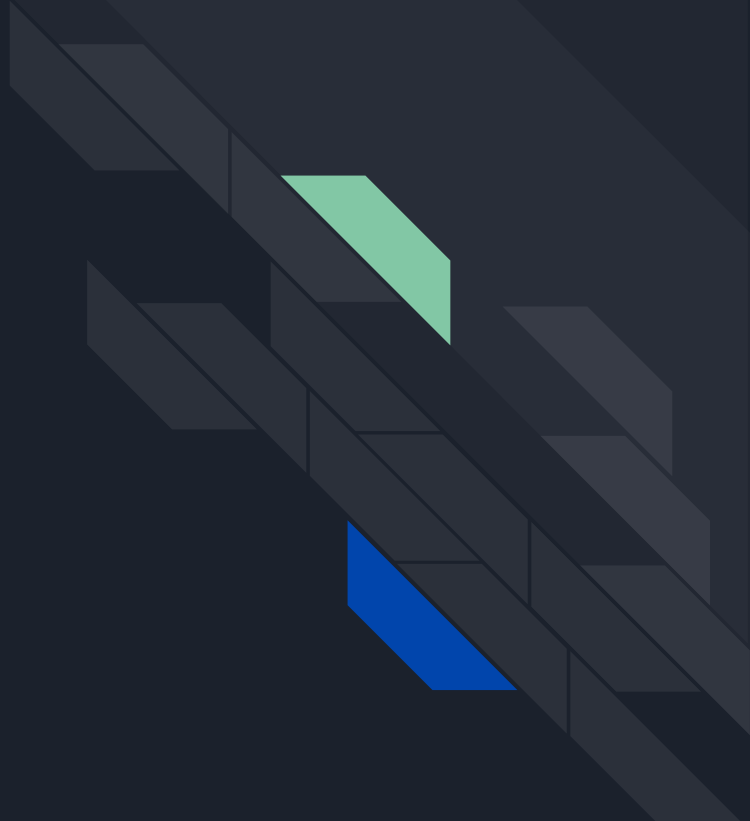
Ejemplos de Optimización (x86)



Ejemplos de Optimización (ARM)



Gracias!

An abstract graphic on the right side of the slide, consisting of several dark gray rectangular blocks arranged in a staggered, 3D-like pattern. Two blocks are highlighted: one in a light green color and one in a blue color, both positioned towards the bottom right of the arrangement.